

***IntelliCar*: A Hybrid Voice-Controlled Rover Combining Low Latency and Natural Language Understanding**

ECE 655L Final Project Report

Yihang Zou, Yuchu Guo, Yifan Xu

Duke University

{yz897, yg246, yx277}@duke.edu

December 6, 2025

Abstract

Abstract—This project presents *IntelliCar*, a voice-controlled rover designed to solve the trade-off between local processing latency and cloud-based natural language understanding. Existing voice control systems typically suffer from either the high latency of cloud APIs or the rigid command structure of local keywords. To address this, we developed an adaptive multi-path pipeline running on a Raspberry Pi 4B. The system integrates Vosk for zero-latency keyword spotting, OpenAI’s LLM for complex intent analysis, and a local Whisper fallback for high-fidelity transcription. Our experimental results demonstrate that the system achieves a command-to-action latency of merely 38ms for local grammar commands, while successfully handling complex natural language queries with a fallback latency of approximately 5.5s. Furthermore, we implemented a closed-loop motion control system using an MPU-6050 gyroscope to ensure precise execution.

1 Introduction and Motivation

1.1 The Voice Control Dilemma

In the domain of robotics and autonomous vehicles, voice interaction is a critical interface. However, developers currently face a “Voice Control Dilemma”:

- **Cloud-Only Solutions:** These offer powerful understanding and context awareness (e.g., ChatGPT, Alexa). However, they introduce high latency and depend heavily on internet stability, making them dangerous for real-time navigation.
- **Local-Only Solutions:** These provide extremely fast execution (often in milliseconds) but suffer from rigid command structures. They fail to understand natural, conversational language or complex instructions.

1.2 Project Mission

Our mission is to create an embedded system that combines the low latency of local processing with the natural language understanding of Cloud AI. By implementing a hybrid architecture on the *IntelliCar* rover, we aim to ensure safety through fast reaction times while maintaining user-friendly interaction.

2 System Design

2.1 Hardware Architecture

The *IntelliCar* (Fig. 1) is built upon a robust hardware platform designed for mobility and computation:

- **Compute Unit:** Raspberry Pi 4B (8GB RAM) [1] handles the orchestration of the VAD, STT, and control logic.
- **Sensors:** A USB Conference Microphone captures audio, while an MPU-6050 Gyroscope [2] provides real-time feedback for precise turning.
- **Actuation:** An L298N Dual H-Bridge Motor Driver controls the DC motors.
- **Power Supply:** A dual-power system featuring a 5V/3A power bank for the Pi and a 12V battery pack for the motors.

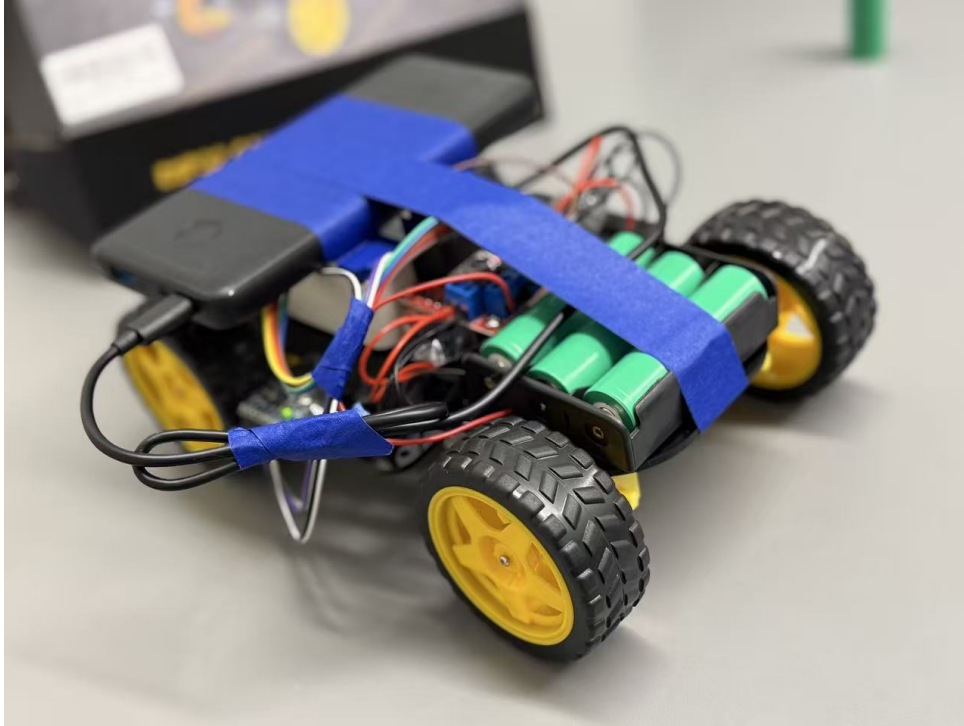


Figure 1: The design of the *IntelliCar*

2.2 Software Architecture

The system is implemented in Python, leveraging concurrency to manage input and output streams.

- **Voice Recognition:** Vosk API [3] for lightweight keyword spotting; Faster-Whisper [4] for fall-back transcription.
- **Natural Language Processing:** Using GPT-4o/gpt-oss-120b model by calling OpenAI/Cerebras API [5, 6] for intent extraction and JSON generation.
- **Motion Control:** Custom PID-like logic integrating MPU-6050 data.

3 System Implementation

3.1 Original Two-Path Design

The initial version of the *IntelliCar* system followed a straightforward two-path architecture, illustrated in Fig. 2. At this stage, we assumed that voice interaction needs could be cleanly separated into two categories: *fast, deterministic keyword commands* and *slower, more expressive natural-language instructions*. This led to the design of a local keyword pipeline and a cloud-based semantic pipeline.

3.1.1 Local Path: Vosk → JSON

In the original design, the Raspberry Pi relied solely on the lightweight Vosk keyword-spotting engine to detect a predefined set of commands (e.g., “forward”, “stop”, “turn left”). When the confidence exceeded the threshold (0.85 in the proposal), the recognized text was directly converted into a JSON command and sent to the motor controller. This achieved very low latency but was limited to a fixed and rigid vocabulary.

3.1.2 Cloud Path: Whisper → LLM → JSON

Commands that were not reliably recognized by Vosk were routed to a cloud-based pipeline. The audio was transcribed using Whisper and then passed to the OpenAI API for semantic parsing into structured

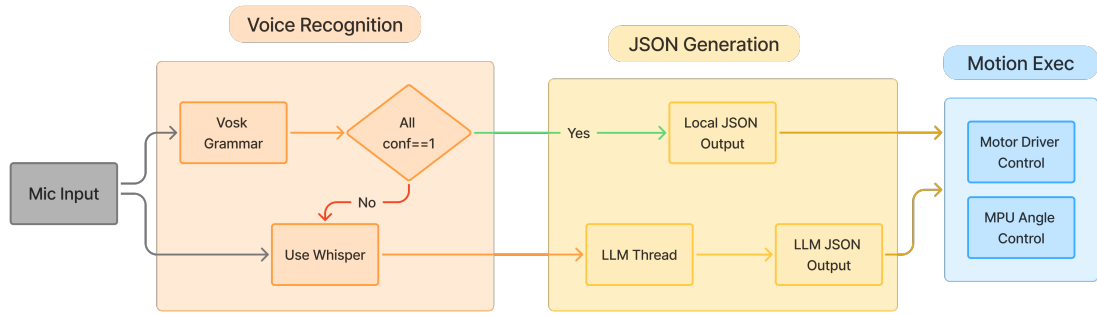


Figure 2: Overview of the *IntelliCar* original 2-path system architecture.

JSON actions. While this allowed complex, conversational instructions to be understood, Whisper’s multi-second inference time on the Raspberry Pi introduced substantial latency, making the cloud path unsuitable for time-sensitive control.

3.2 Challenges Encountered During Development

During real-world testing on the Raspberry Pi 4B, several limitations of the original two-path design became apparent. Each issue directly motivated the architectural changes leading to the adaptive system described in the next subsection.

Challenge 1: Whisper Latency on the Raspberry Pi

Whisper inference on the Raspberry Pi required several seconds per segment, making the original cloud path unusable for interactive or safety-critical commands.

Solution: Introduce a Vosk No-Grammar model and allow direct LLM parsing without Whisper, forming the basis of the later “Direct Path”.

Challenge 2: VAD Failure in Noisy Environments

Energy-based VAD frequently failed under ambient noise, causing oversized audio segments and further increasing Whisper latency.

Solution: Replace VAD with a ring-buffer segmentation mechanism driven by Vosk token time-stamps, ensuring clean and bounded audio extraction regardless of noise conditions.

Challenge 3: Unsafe or Ambiguous LLM Outputs

LLM responses occasionally produced unbounded or unsafe motion parameters.

Solution: Enforce strict JSON schema validation (angle, duration, speed limits), rejecting any action that falls outside well-defined control constraints.

Challenge 4: Concurrency and System Freezes

Concurrent Whisper and LLM operations occasionally caused thread blocking and, in extreme cases, temporary freezes due to uncontrolled queue growth.

Solution: A dedicated `throttle_worker` monitors active LLM tasks against `LLM_ENQUEUE_THRESHOLD`, ensuring that audio and command queues remain bounded and preventing resource exhaustion on the Raspberry Pi.

Challenge 5: Inaccurate Turning in Open-Loop Control

Time-based turning exhibited significant angular drift due to motor asymmetry and mechanical inertia.

Solution: Integrate MPU-6050 gyroscope feedback and implement an adaptive stop-guard correction mechanism, enabling reliable closed-loop angle control.

1. **Closed-Loop Gyro Control:** For turning actions, the system integrates Z-axis angular velocity from the MPU-6050 sensor to track heading in real-time.
2. **Adaptive Stop Guard:** To counteract mechanical inertia, the system implements a dynamic parameter `stop_guard_dynamic`. This value is automatically tuned based on the overshoot or undershoot of previous turns ($\pm 10^\circ$ adjustment range), ensuring precise rotations over time.
3. **Settle Logic:** After motor braking, the system waits for the angular velocity to drop below a threshold (0.05 rad/s) to ensure the device is fully stable before accepting the next command.

3.3 Adaptive Multi-Path Pipeline

Building upon the limitations identified in the original two-path design, the final implementation adopts an adaptive multi-path pipeline (Fig. 3) that dynamically selects among three processing routes. As implemented in `vosk_whisper_llm.py`, the system routes audio data through one of three paths based on model confidence and transcript consistency:

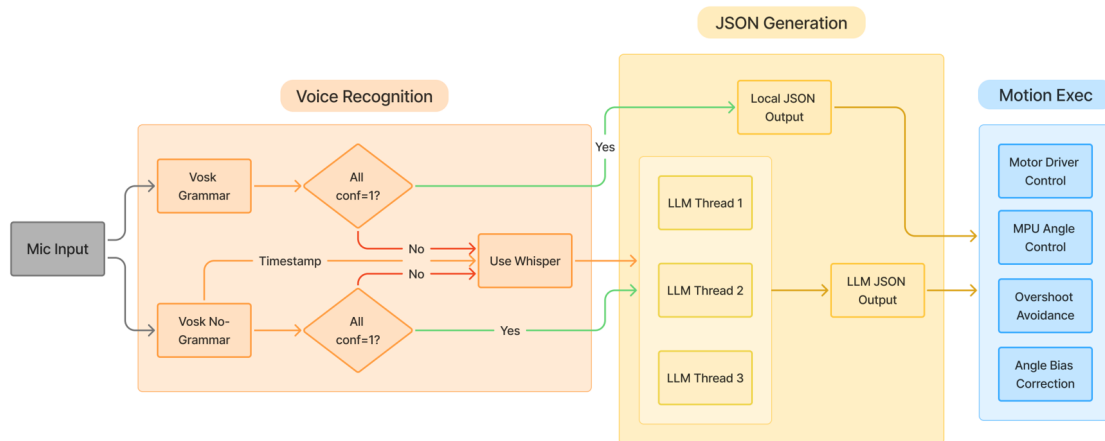


Figure 3: Overview of the *IntelliCar* adaptive multi-path system architecture.

3.3.1 Fast Path (Deterministic Local Parsing)

Triggered when the Vosk Grammar model detects a perfect match (Confidence = 1.0) and the transcript matches the restricted vocabulary defined in `vosk_config.py`.

- **Mechanism:** The text is parsed by a deterministic Regex-based parser (`parse_en.py`) to immediately generate structured commands.
- **Latency:** Minimal ($< 200\text{ms}$), bypassing all neural network inference for simple commands like “Stop” or “Turn Left”.
- **Flow:** Mic $\rightarrow$ Vosk Grammar $\rightarrow$ Regex Parser $\rightarrow$ Motion.

3.3.2 Direct Path (Hybrid LLM)

If the Grammar model fails but the No-Grammar model produces a coherent natural-language transcript, the system sends the text directly to the LLM for schema-constrained interpretation. This path leverages the semantic understanding of the LLM without the overhead of a second STT pass.

Flow: Mic $\rightarrow$ Vosk Natural $\rightarrow$ LLM (Schema Enforcement) $\rightarrow$ Motion.

3.3.3 Whisper Fallback Path

This is the most robust path, triggered when both Vosk Grammar & No-Grammar model have low confidence. The system utilizes a ring buffer to retrospectively extract the relevant audio segment (adding padding via WHISPER_PAD_SECONDS). The audio is re-transcribed by a local instance of Faster-Whisper before querying the LLM.

Flow: Mic $\rightarrow$ Audio Ring Buffer $\rightarrow$ Whisper [4] $\rightarrow$ LLM $\rightarrow$ Motion.

4 Experimentation and Evaluation

4.1 Latency Analysis

We conducted rigorous testing to measure the latency of each execution path. The results are summarized in Table 1.

Table 1: Latency Comparison of Execution Paths

Path Type	Description	Total Latency
Fast Path	Local Vosk Grammar $\rightarrow$ Action	0.038 s
Direct Path	Vosk Natural $\rightarrow$ LLM	0.650 s
Fallback Path	Vosk $\rightarrow$ Whisper $\rightarrow$ LLM	5.580 s

Analysis:

- The **Fast Path** achieves extremely low latency ($\approx 38\text{ms}$), making it ideal for safety-critical commands like "Stop".
- The **Direct Path** offers a balanced trade-off (0.65s), suitable for most natural language interactions.
- The **Fallback Path** is significantly slower (5.58s) due to the heavy computation of Whisper transcription and LLM network overhead, but it provides the highest reliability in noisy conditions.

4.2 Qualitative Results

The system successfully demonstrated "overshoot avoidance" and "angle bias correction" during the demo. The integration of the Gyroscope allowed the car to turn precisely to requested angles (e.g., "turn 90 degrees") rather than relying on estimated time-based turning.

5 Discussions and Conclusions

5.1 Discussion

The *IntelliCar* system demonstrates that a hybrid architecture can effectively solve the "Voice Control Dilemma" by balancing execution speed with semantic understanding.

5.1.1 Performance Analysis and Trade-offs

Our latency analysis highlights a massive performance gap between local signal processing and neural transcription. The **Fast Path** achieved a latency of just 38ms, validating the decision to use deterministic regex parsing (`parse_en.py`) for critical safety commands like "Stop". In contrast, the **Whisper Fallback Path** incurred a total latency of 5.58s, with the Whisper transcription process alone consuming

5.007s (approx. 90% of the total time). However, the **Direct Path** serves as an effective middle ground, offering natural language capabilities via the LLM with a manageable latency of 0.65s. This multi-path approach ensures that the system is not penalized by the heavy computational cost of Whisper unless the acoustic environment or command complexity strictly demands it.

5.1.2 Safety and Determinism

While Large Language Models provide flexibility, they are prone to hallucination. Our implementation successfully mitigated this risk through a strict JSON Schema enforcement strategy. As detailed in the system architecture, the LLM is constrained to output only valid fields ('action', 'duration_ms', 'angle_deg', 'speed'). This ensures that vague inputs are either rejected or converted into safe, bounded execution parameters before reaching the motor driver.

5.1.3 Hardware Constraints and Scalability

The primary bottleneck identified is the inference speed of the Faster-Whisper model on the Raspberry Pi 4B CPU. While the ring-buffer implementation successfully prevented audio timeouts during these long inference windows, the 5-second delay is suboptimal for real-time navigation. Future iterations could address this by offloading the Whisper inference to a distinct edge accelerator or upgrading to a Raspberry Pi 5, which offers significantly higher CPU throughput for quantized models.

5.1.4 Motion Fidelity

Finally, the integration of the MPU-6050 gyroscope proved essential for bridging the gap between “understanding” and “doing”. The adaptive stop guard logic implemented in the motor bridge allowed the system to dynamically correct for mechanical inertia, ensuring that a command to “turn 90 degrees” resulted in a physical rotation within acceptable error margins ($\pm 5^\circ$), rather than the arbitrary drifts common in open-loop systems.

5.2 Future Roadmap

Building on the current adaptive architecture, future development will target three critical enhancements outlined in our project vision:

1. **Visual Autonomy (Computer Vision):** Currently, the rover operates “blindly” based on voice commands. We plan to integrate a Raspberry Pi Camera module with OpenCV to enable semantic commands such as “Follow the red ball”. This would allow the system to combine audio intent with visual tracking for closed-loop navigation.
2. **Contextual Memory & Mapping:** The current LLM implementation in `vosk_whisper_llm.py` treats each command independently. Future iterations will implement a conversation history buffer (Multi-turn Context), allowing the rover to understand relative commands like “Go back to the previous location” or semantic destinations like “Take me to the kitchen”.
3. **Edge Optimization & Power Efficiency:** To address the 5.58s latency observed in the Whisper **Fallback Path**, we aim to optimize the inference pipeline. This includes exploring dedicated hardware accelerators (e.g., Coral Edge TPU) to offload the `faster-whisper` model and implementing rigorous sleep modes to extend the battery life of the 12V power pack.

5.3 Conclusion

IntelliCar successfully demonstrates a viable solution to the “Voice Control Dilemma” by orchestrating a hybrid edge-cloud architecture. By dynamically routing audio between a millisecond-level local parser and a high-fidelity LLM pipeline, the system achieves the reliability of embedded controls with the flexibility of modern Generative AI.

While hardware limitations on the Raspberry Pi 4B present latency challenges for heavy models, the implementation of concurrent threading, ring-buffer audio management, and strict schema validation proves that robust voice navigation is achievable on commodity hardware. This architecture paves the way for practical applications in assistive indoor mobility carts and hands-free industrial inventory robots, where both safety and natural interaction are paramount.

References

- [1] Raspberry Pi Foundation, *Raspberry Pi 4 Model B Datasheet*, June 2019. Accessed: 2025-12-04.
- [2] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification Revision 3.4*, August 2013. Accessed: 2025-12-04.
- [3] Alpha Cephei, “Vosk offline speech recognition api.” <https://alphacephei.com/vosk/>, 2023. Accessed: 2025-12-04.
- [4] SYSTRAN, “Faster-whisper: Faster whisper transcription with ctranslate2.” <https://github.com/SYSTRAN/faster-whisper>, 2023. Accessed: 2025-12-04.
- [5] OpenAI, “Openai api documentation.” <https://platform.openai.com/docs/>, 2024. Accessed: 2025-12-04.
- [6] C. Systems, “Build with cerebras inference — introduction.” <https://inference-docs.cerebras.ai/introduction>, 2025. Accessed: 2025-12-05.